

# Phase Box: Combined Technical Specification & Architecture Decision Record (ADR)

This document serves as the definitive technical reference and Claude Agent implementation guide for building the Phase Box algorithmic MIDI modulation engine.

## 1. Feature Summary

- Algorithmic Modulation:** Generates continuous, non-repetitive control streams (Sine, Triangle, Square, S&H).
- Nested LFO Engine:** Background LFOs modulate the frequency/depth of foreground LFOs for evolving organic drifts.
- Turing-Machine Mutations:** Probabilistic step mutation for generative, non-repeating rhythmic patterns.
- Multi-Destination Macros:** Expression pedal mapped to weighted multi-parameter scaling matrices.
- Tactile Control Matrix:** Live-first interface using an infinite rotary encoder, dedicated mode buttons, and dual footswitches.
- Zero-Friction Preset Workflow:** "Hold Encoder to Save" captures live tweaks instantly to onboard flash via LittleFS.
- Multi-Channel MIDI:** Independent CC routing across multiple channels (CH1: Auto Waaaves, CH2: r\_e\_c\_u\_r).
- Dual Clock Architecture:** Auto-sync to incoming external MIDI Clock with seamless fallback to Tap Tempo and instant Phase Reset.
- Power Topology:** Operates on standard 9V DC pedalboard power, regulated internally to 5V for the Pico.
- Host-Side Development:** Core logic compiles on Ubuntu via a Hardware Abstraction Layer (HAL) for simulation prior to hardware flashing.

## 2. Architecture Decision Record (ADR)

### 2.1. Context & Problem Statement

We are building a custom hardware device, "Phase Box," based on a Raspberry Pi Pico. Its primary function is to act as an algorithmic modulation engine, continuously generating MIDI Control Change (CC) messages to control parameters on two external video synthesis applications: **Auto Waaaves** and **r\_e\_c\_u\_r**. The device must be gig-ready, robust, and feature an intuitive live-first interface that allows capturing accidental or deliberate parameter tweaks instantly.

### 2.2. Decision Drivers

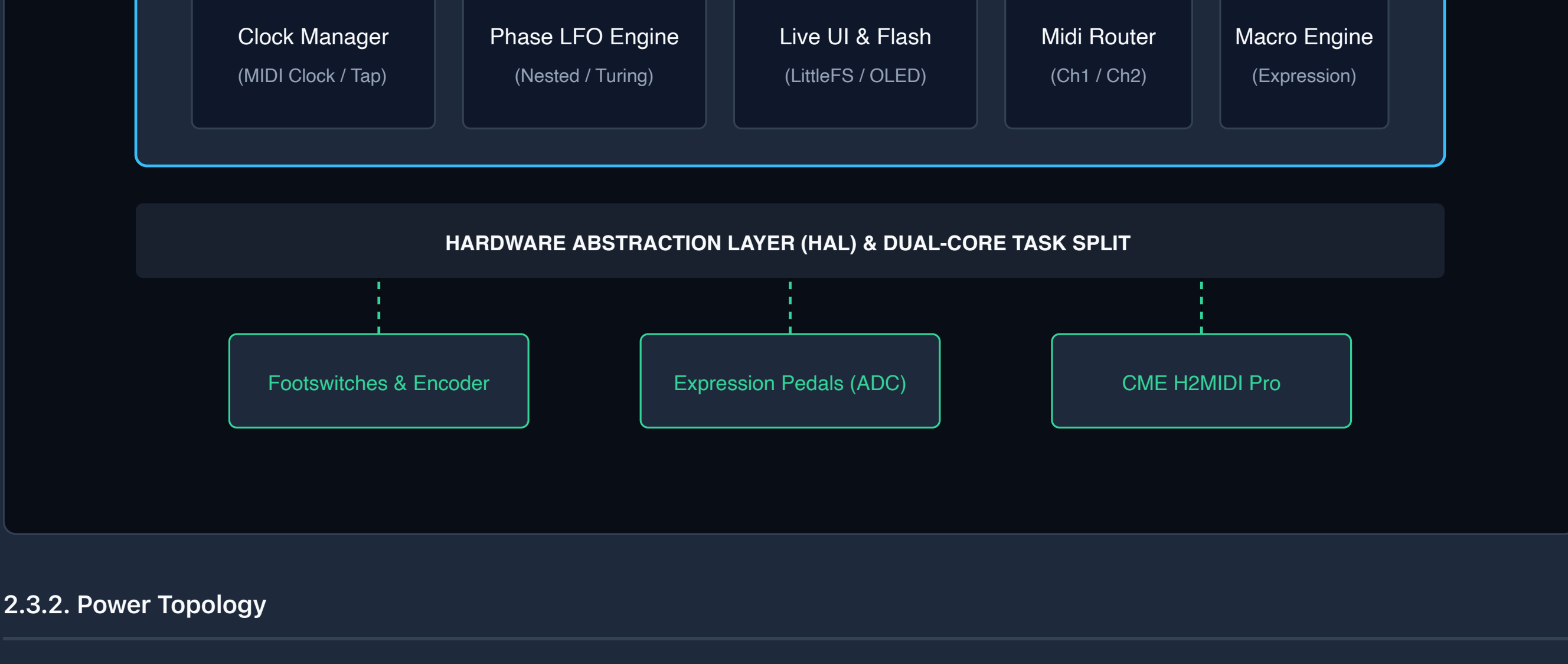
- Live Reliability & Determinism:** The core algorithmic generation (LFOs, S&H) must not glitch or stutter. Latency must be minimal.
- Dual Clock Sovereignty:** Must auto-sync to an incoming MIDI Clock signal but fallback seamlessly to internal Tap Tempo when the clock stops. An instant "Phase Reset" footswitch is critical for rhythmic alignment.
- Zero-Friction Preset Capture:** The interface must support saving the "current live state" to flash memory via a simple, long-press gesture on the encoder ("Hold-to-Save"), without navigating menus.
- Multi-Target & Nested Routing:** Must partition independent modulation streams across multiple MIDI channels and support routing slower LFOs to modulate faster LFOs.
- Power Isolation:** Must operate cleanly off standard pedalboard 9V DC power supplies, isolated from USB bus power to prevent ground loops and noise artifacts.
- Developer Velocity:** Must allow the core C++ logic (math, routing, state machines) to be compiled, simulated, and unit-tested on a Linux host (Ubuntu) without requiring the microcontroller hardware attached.

### 2.3. Architectural Decisions

#### 2.3.1. Modular Software & Hardware Abstraction Layer (HAL)

The firmware is strictly decoupled into self-contained modules (e.g., `PhaseLFO`, `ClockManager`, `InputMatrix`). All hardware dependencies are abstracted behind clean C++ interfaces defined in a HAL. This enables two concrete implementations:

- `MockImplementation`: Links against standard C++ and testing frameworks (GoogleTest) on Ubuntu for simulation.
- `PicoImplementation`: Uses the `pico-sdk` to interact with the actual RP2040 hardware.



#### 2.3.2. Power Topology

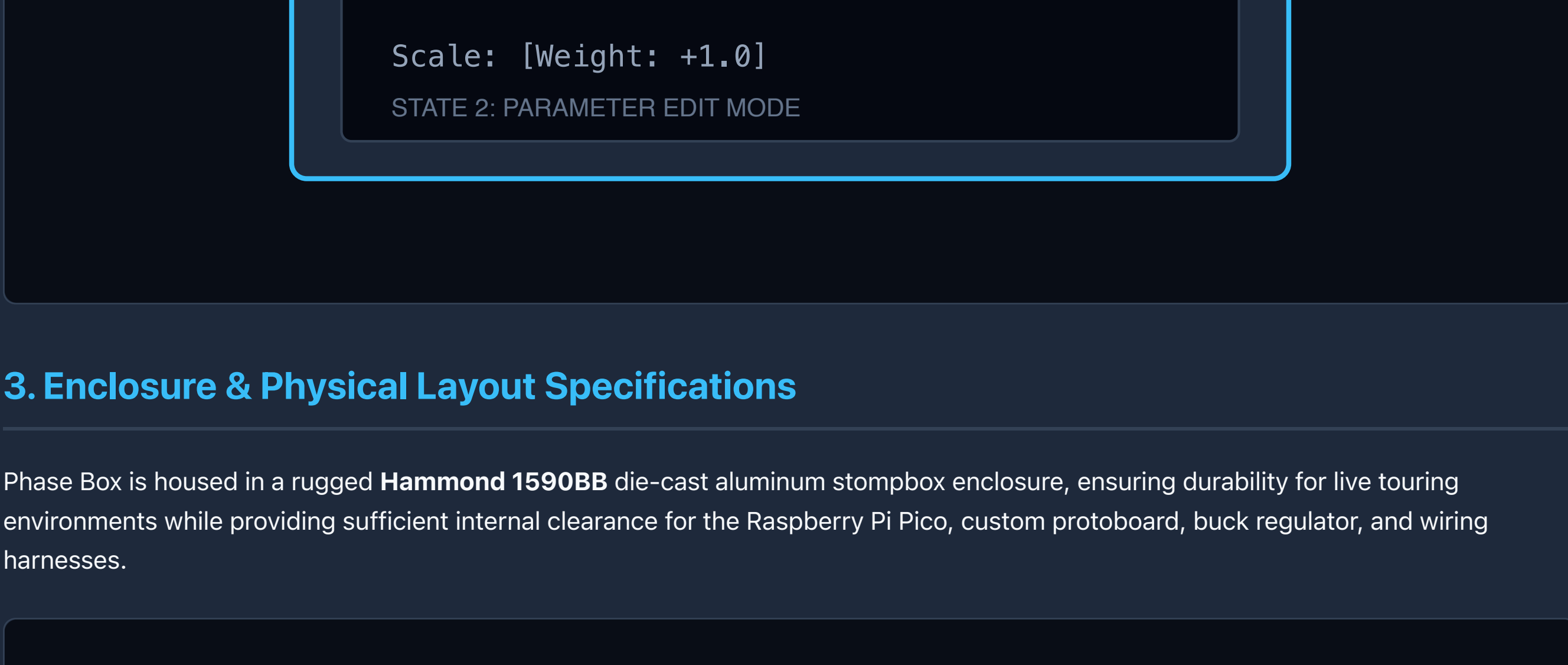
To ensure galvanic isolation and eliminate digital noise, the Phase Box operates on standard 9V DC pedalboard power. This voltage is stepped down internally to 5V via a dedicated buck regulator connected to the Pico's `V5VS` pin. The Pico's USB port is used exclusively for flashing and debugging, not for live performance power.

#### 2.3.3. Preset Storage & Data Persistence

Presets are stored in the RP2040's onboard SPI NOR flash memory using the LittleFS filesystem, mapping structured binary configuration blobs to virtual files (e.g., `patch1.bin`).

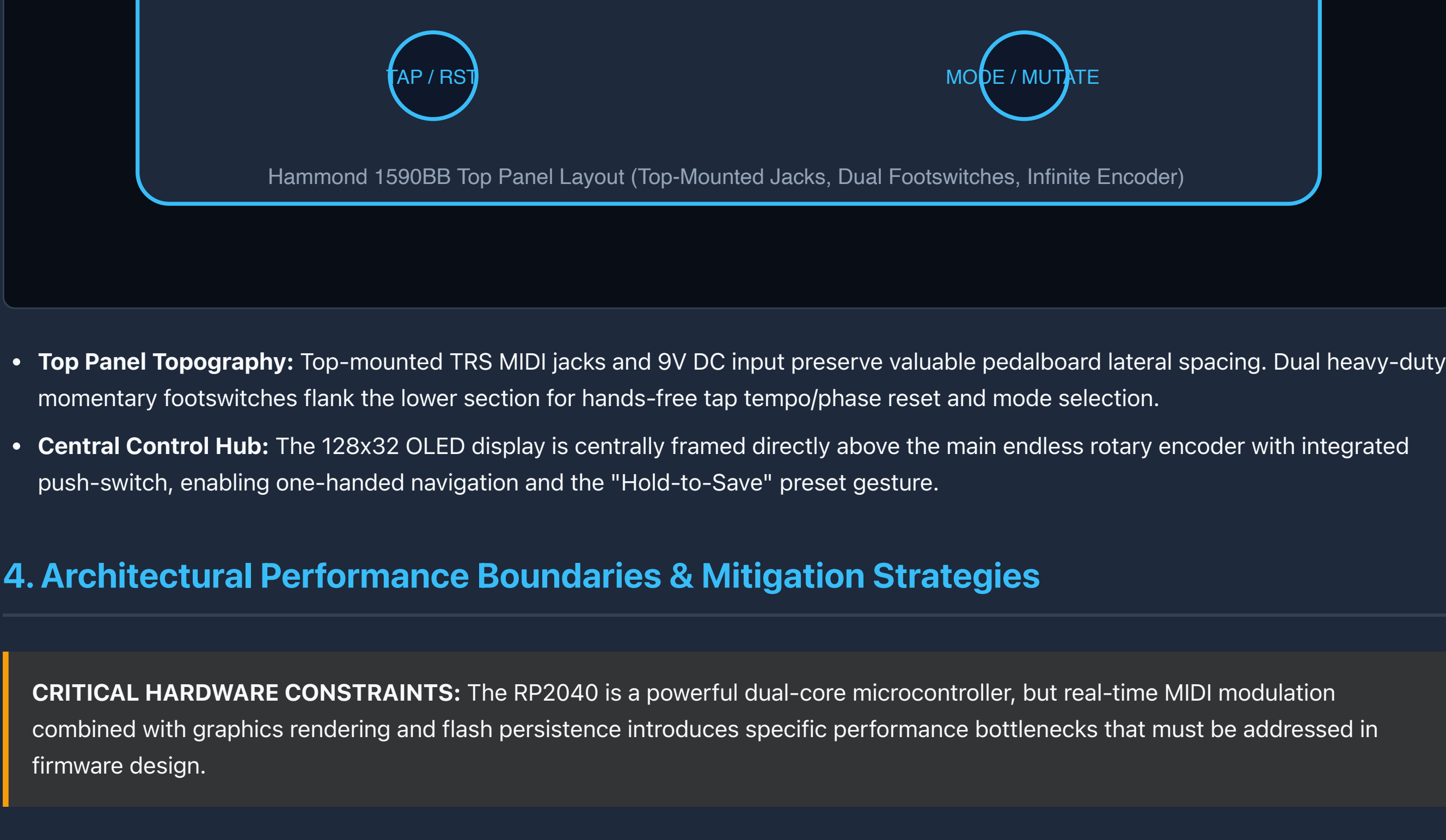
### 2.4. Interactive OLED Workflow Simulation

The animated schematic below cycles through the four primary live performance states rendered on the 128x32 I2C OLED display:



## 3. Enclosure & Physical Layout Specifications

Phase Box is housed in a rugged **Hammond 1590BB** die-cast aluminum stompbox enclosure, ensuring durability for live touring environments while providing sufficient internal clearance for the Raspberry Pi Pico, custom protoboard, buck regulator, and wiring harnesses.



- Top Panel Topography:** Top-mounted TRS MIDI jacks and 9V DC input preserve valuable pedalboard lateral spacing. Dual heavy-duty momentary footswitches flank the lower section for hands-free tap tempo/phase reset and mode selection.
- Central Control Hub:** The 128x32 OLED display is centrally framed directly above the main endless rotary encoder with integrated push-switch, enabling one-handed navigation and the "Hold-to-Save" preset gesture.

## 4. Architectural Performance Boundaries & Mitigation Strategies

**CRITICAL HARDWARE CONSTRAINTS:** The RP2040 is a powerful dual-core microcontroller, but real-time MIDI modulation combined with graphics rendering and flash persistence introduces specific performance bottlenecks that must be addressed in firmware design.

### 4.1. Floating-Point Math Optimization (No Hardware FPU)

- The Boundary:** The RP2040 dual Cortex-M0+ cores lack a hardware Floating-Point Unit (FPU). Heavy usage of raw floating-point operations in tight loops introduces CPU overhead and modulation jitter.
- The Solution:** Pre-calculate lookup tables (LUTs) for standard waveforms (Sine, Triangle) instead of computing trigonometric functions on the fly. Utilize integer or fixed-point math for phase accumulator increments where appropriate.

### 4.2. I2C Bus Saturation Prevention

- The Boundary:** Blasting pixel updates to the 128x64 OLED display over a standard I2C bus too frequently blocks the main thread, causing MIDI clock drift and dropped events.
- The Solution:** Run the I2C bus at Fast-Mode Plus (1MHz) and decouple the display loop from the audio/MIDI loop. Throttle the OLED render routine to a stable 30 FPS using delta-time checks.

### 4.3. Dual-Core Asymmetric Task Splitting

- The Boundary:** Handling USB/MIDI interrupts, LFO math, ADC macro scaling, and OLED rendering on a single execution thread causes timing inconsistencies.
- The Solution:** Distribute workloads across the RP2040's dual cores:
  - Core 0 (High Priority):** Dedicated entirely to real-time engine tasks—MIDI clock processing, `PhaseLFO` math, Turing machine mutations, macro scaling, and message dispatching.
  - Core 1 (Background Priority):** Handles lower-priority operations—LittleFS flash read/writes, button input matrix debouncing, and OLED display rendering.

### 4.4. Flash File System (LittleFS) Write Isolation

- The Boundary:** Writing presets to flash memory triggers page erasure cycles that temporarily block execution on the active core, risking micro-stutters during live play.
- The Solution:** Isolate all flash read/write operations exclusively to Core 1 and restrict them to explicit user actions (e.g., the 1-second encoder "Hold-to-Save" gesture), ensuring zero interference with real-time MIDI generation on Core 0.

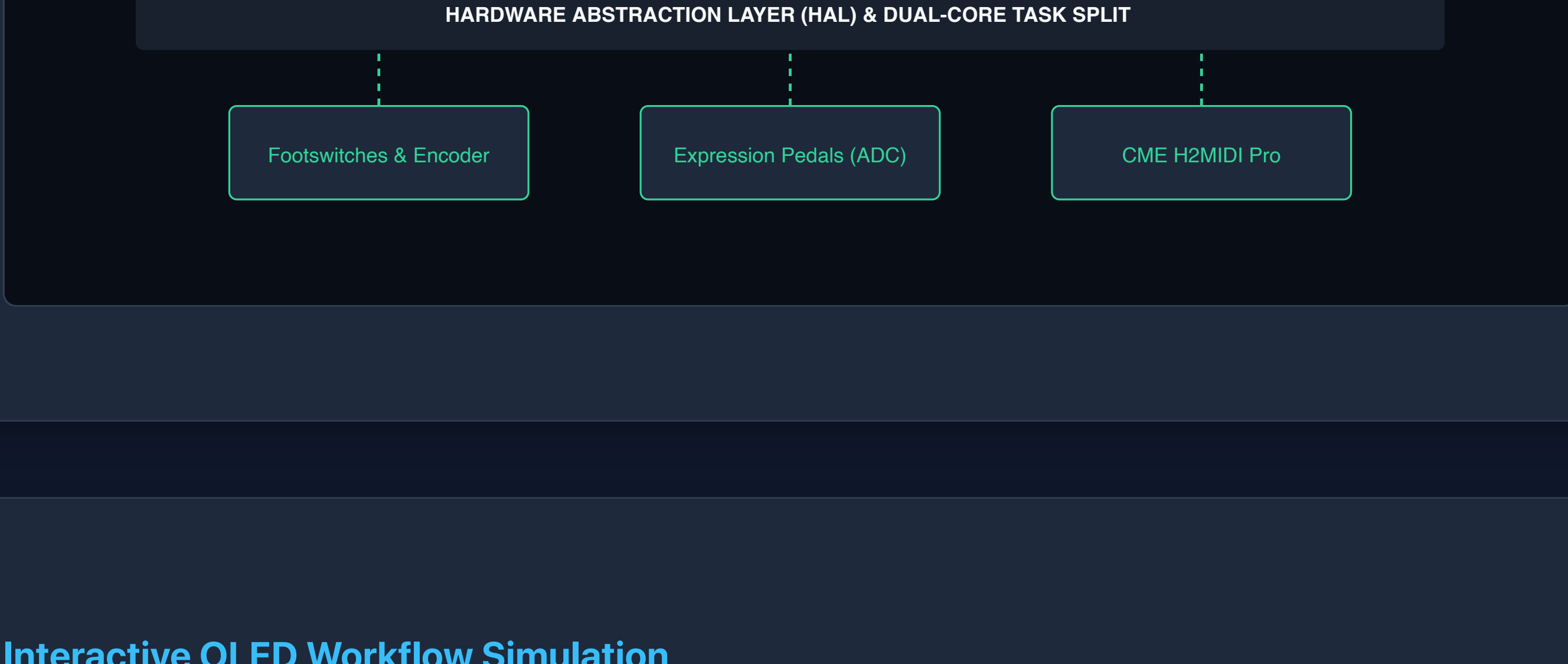
## Presentation Showcase

# Phase Box: Algorithmic MIDI Modulation Engine

A robust hardware and software technical specification showcase for gig-ready video synth modulation powered by the Raspberry Pi Pico.

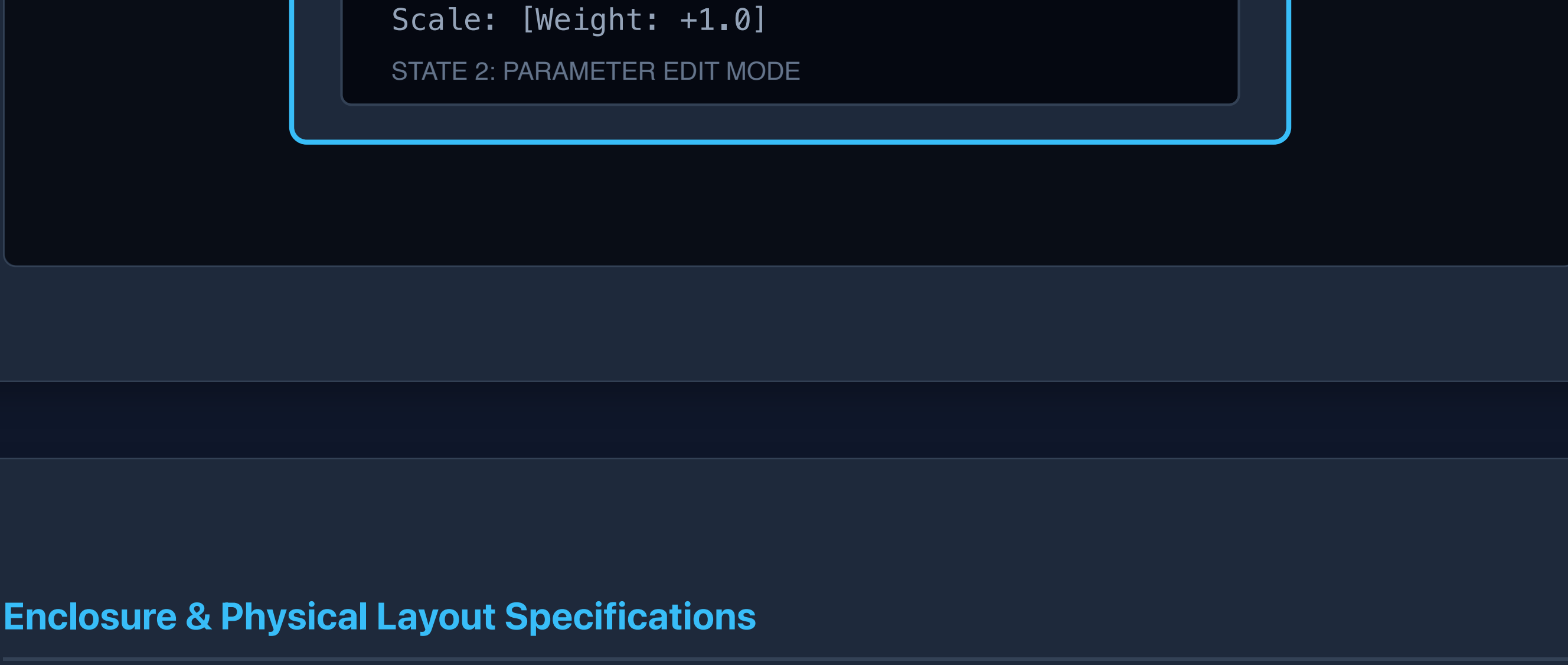
## Core Concepts & Architecture

Modular design separating core math from target hardware for seamless Ubuntu simulation and RP2040 deployment.



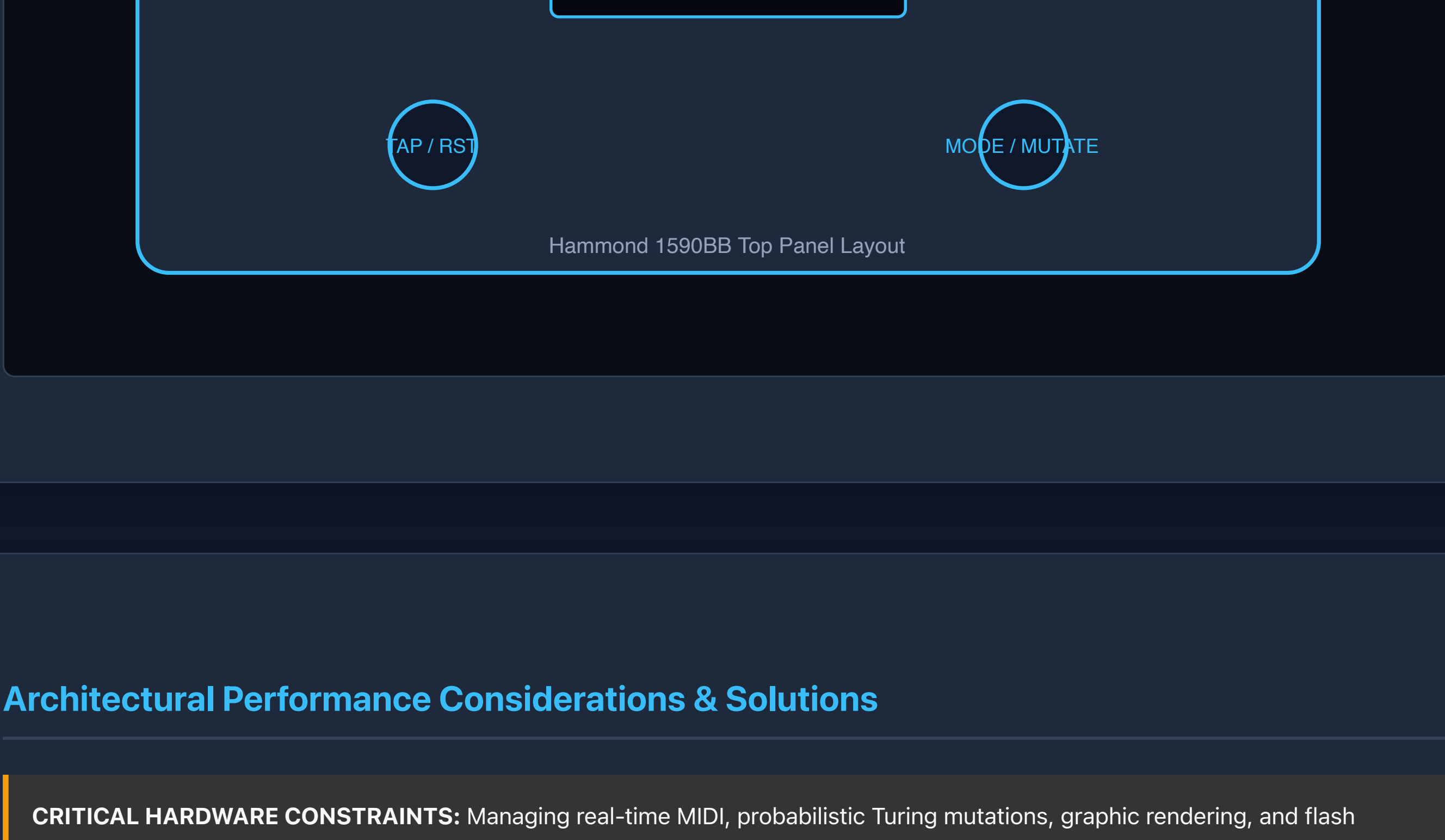
## Interactive OLED Workflow Simulation

Visual states during live performance handling telemetry, parameter edits, preset saves, and clock syncing.



## Enclosure & Physical Layout Specifications

Hammond 1590BB enclosure design ensuring touring durability and ergonomic pedalboard access.



## Architectural Performance Considerations & Solutions

**CRITICAL HARDWARE CONSTRAINTS:** Managing real-time MIDI, probabilistic Turing mutations, graphic rendering, and flash persistence on the RP2040.

- Floating-Point Math (No Hardware FPU):** The RP2040 Cortex-M0+ lacks a hardware FPU. *Solution:* Pre-calculate lookup tables (LUTs) for standard waveforms (Sine, Triangle) and use fixed-point arithmetic for phase accumulators.
- I2C Bus Saturation:** Blasting pixel data to the OLED can block the main thread and cause MIDI jitter. *Solution:* Run I2C at Fast-Mode Plus (1MHz) and throttle display rendering to a stable 30 FPS.
- Dual-Core Asymmetric Task Splitting:** *Solution:* Core 0 is dedicated to real-time engine processing (MIDI clock, LFO math, Turing mutations, macro scaling), while Core 1 handles low-priority tasks (LittleFS flash writes, button debouncing, OLED rendering).
- Flash File System (LittleFS) Isolation:** Flash write cycles cause blocking delays. *Solution:* Isolate all write tasks strictly to Core 1 on user demand (Hold-to-Save gesture), completely protecting the Core 0 real-time MIDI output.